

 What is Random
Forest?

Supervised = We have the answer while training.

Decision Tree = Learns from those answers.

Random Forest = Many Decision Trees learn from those answers

Imagine you want to know:

"Will this student pass the exam?"

Instead of asking one teacher, you ask many teachers.

Each teacher looks at different things:



How many hours did the student study?



How many classes did they attend?



What were their previous marks?



Did they complete their assignments?

Each teacher gives an answer:

Teacher 1 → Pass

Teacher 2 → Pass

Teacher 3 → Fail

Teacher 4 → Pass

Teacher 5 → Pass

Now we take the majority vote.

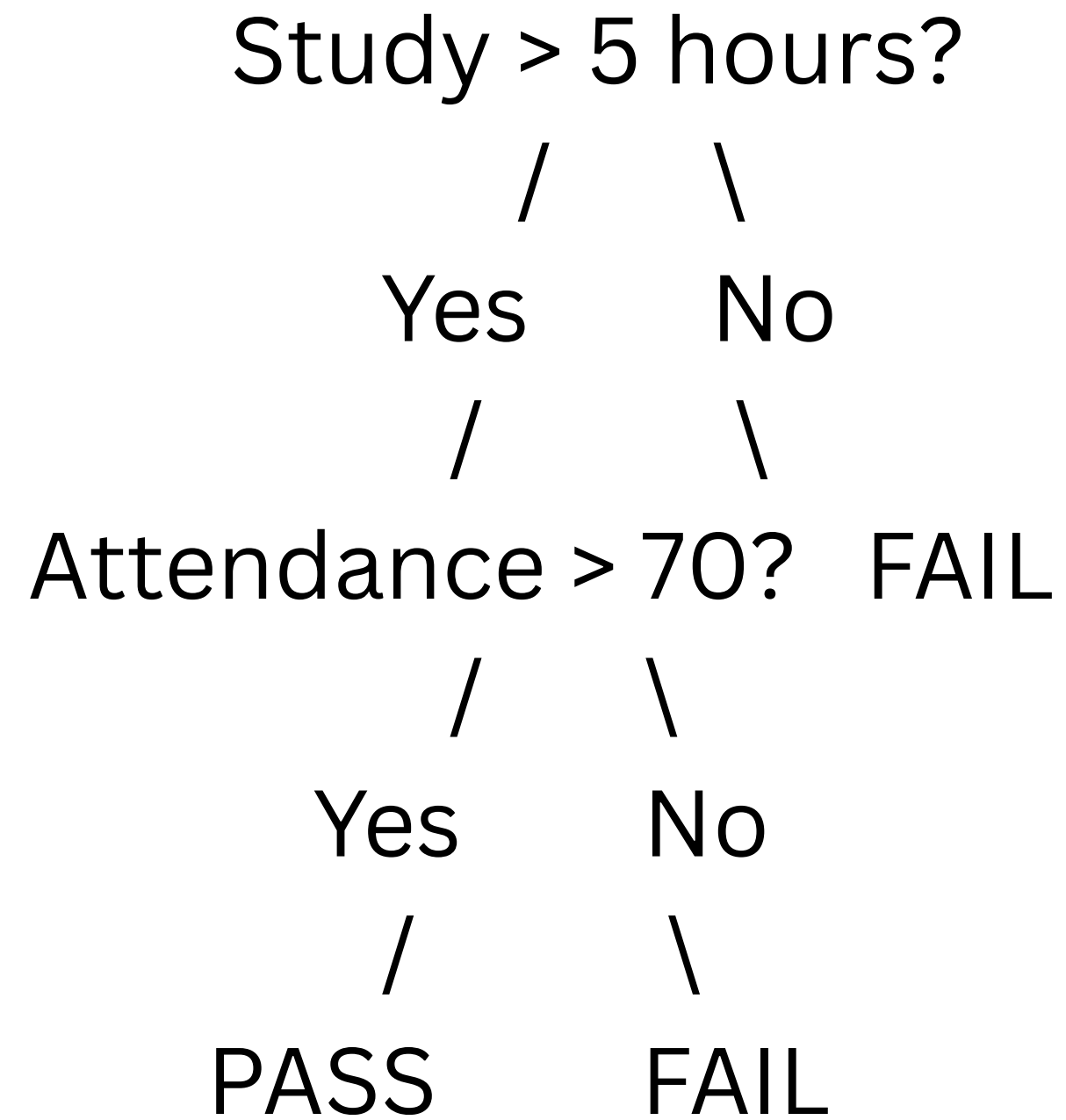
 4 teachers said Pass

 1 teacher said Fail

So Random Forest says:

 PASS

 But where does "Forest" come from?



One tree = 

Decision Tree

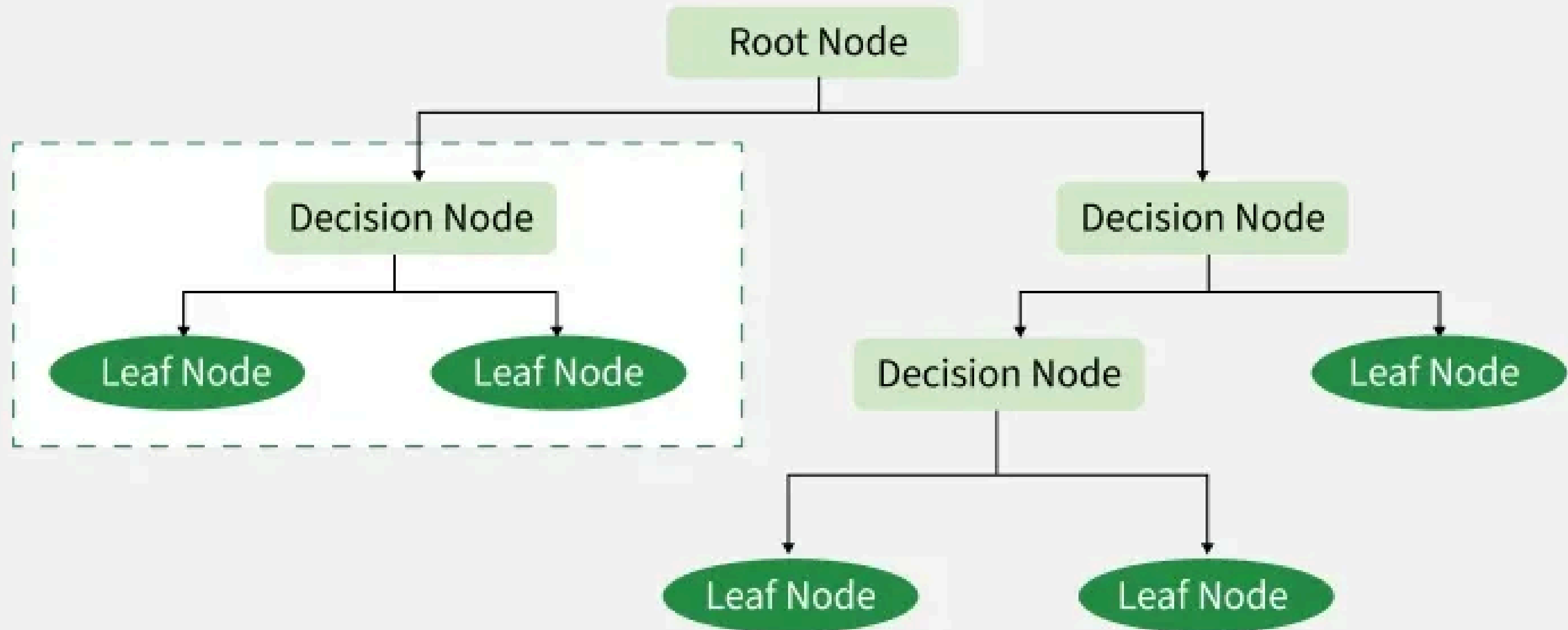
But Random Forest

creates many

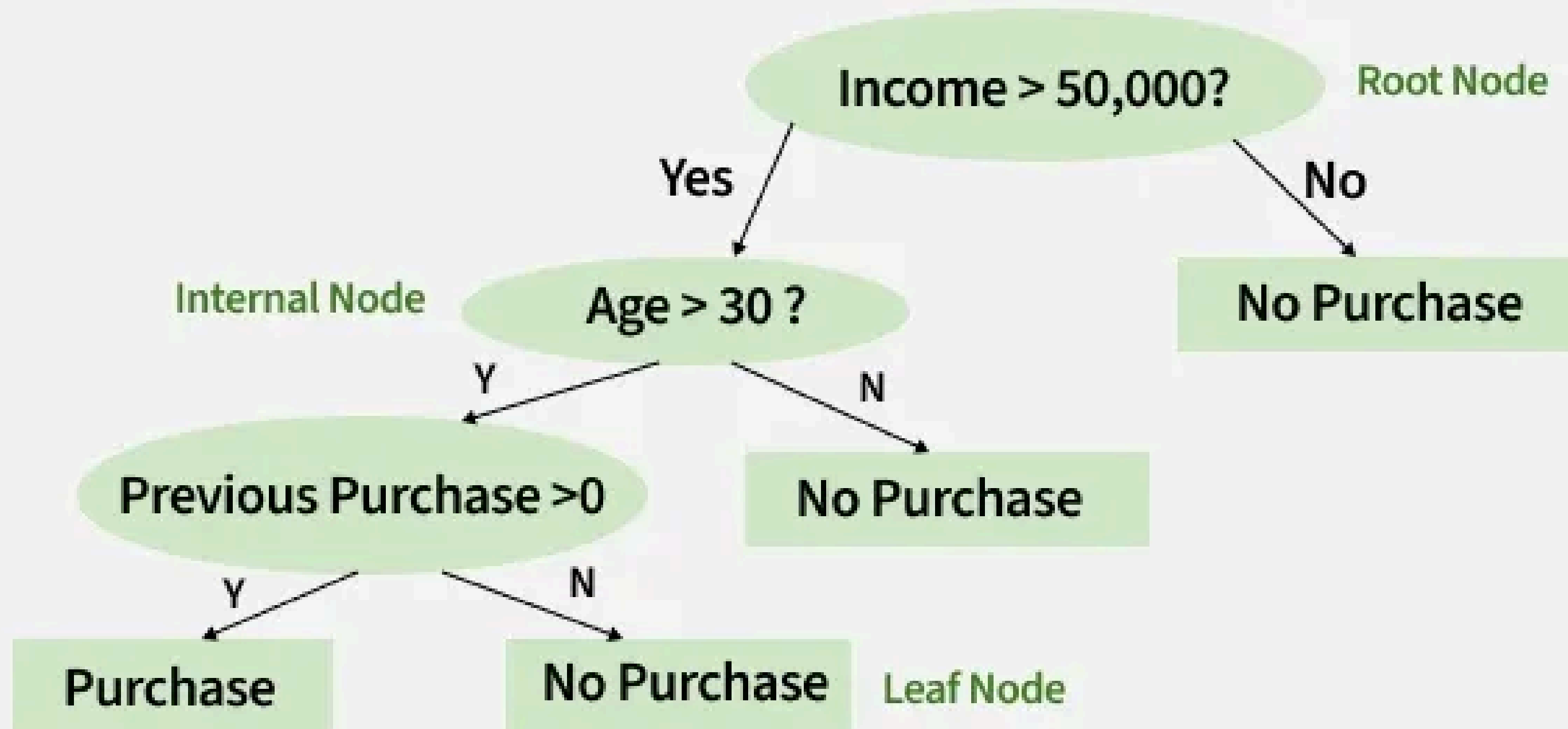
decision trees:

Decision Tree

A decision tree is a flowchart-like model used for classification and regression. It splits data based on features to make predictions . It looks like:



Predicting whether a customer will buy a product



Understanding Decision Tree with Real life use case:

Till now we have understood about the attributes and components of decision tree. Now let's jump to a real life use case in which how decision tree works step by step.

Step 1. Start with the Whole Dataset

We begin with all the data which is treated as the root node of the decision tree.

Step 2. Choose the Best Question (Attribute)

Pick the best question to divide the dataset. For example ask: *"What is the outlook?"*

Possible answers: Sunny, Cloudy or Rainy.

Step 3. Split the Data into Subsets

Divide the dataset into groups based on the question:

- If sunny go to one subset.
- If Cloudy go to another subset.
- If Rainy go to the last subset.

Step 4. Split Further if Needed (Recursive Splitting)

For each subset ask another question to refine the groups. For example If the Sunny subset is mixed ask: *"Is the humidity high or normal?"*

- High humidity → "Swimming".
- Normal humidity → "Hiking".

Step 5. Assign Final Decisions (Leaf Nodes)

When a subset contains only one activity, stop splitting and assign it a label:

- Cloudy → "Hiking".
- Rainy → "Stay side".
- Sunny + High Humidity → "Swimming".
- Sunny + Normal Humidity → "Hiking".

Step 6. Use the Tree for Predictions

To predict an activity follow the branches of the tree. Example: If the outlook is Sunny and the humidity is High follow the tree:

- Start at *Outlook*.
 - Take the branch for Sunny.
 - Then go to Humidity and take the branch for High Humidity.
 - Result: "Swimming".
-

Information Gain	Gini Index
Measures how much information we gain	Measures how mixed the data is
Higher is better	Lower is better
"How useful is this question?"	"How messy is this group?"

Information Gain → "How useful is my question?" 🎯

Gini Index → "How mixed is my group?" 🗑️

"Information Gain tells us how useful the question is."

"Gini Index tells us how mixed the resulting groups are."

"So remember: Information Gain wants to go UP ⬆️, Gini wants to go DOWN ⬇️."



How Decision Tree uses them

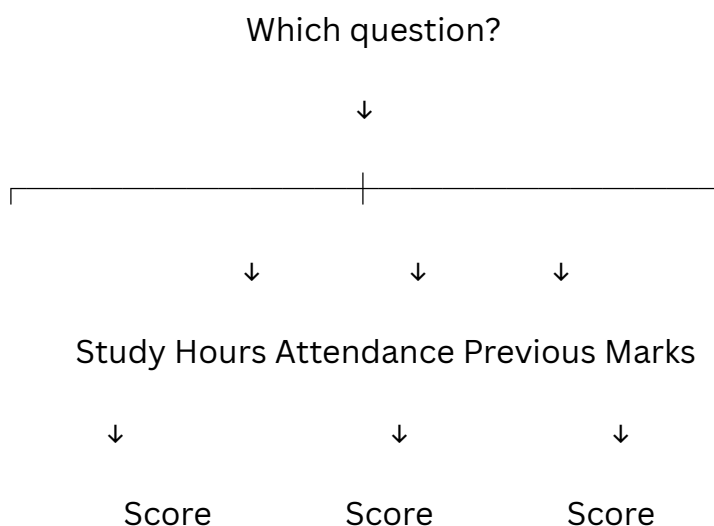
Suppose the tree has three possible questions:

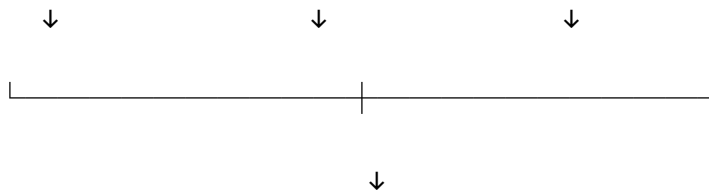
Question 1 → Study Hours?

Question 2 → Attendance?

Question 3 → Previous Marks?

The tree evaluates them.





Choose the best split

If using **Information Gain**:

Choose the split with the **highest Information Gain**.

If using **Gini Index**:

Choose the split with the **lowest Gini impurity**.

$$Gini = 1 - (P_{pass}^2 + P_{fail}^2)$$

8 students 6 Pass 2 Fail

$$P(\text{pass}) = 6/8 = 0.75$$

$$P(\text{fail}) = 2/8 = 0.25$$

$$Gini = 1 - (0.75^2 + 0.25^2)$$

$$= 1 - (0.5625 + 0.0625)$$

$$= 1 - 0.625$$

$$= 0.375$$

Gini = 0.375

The closer Gini gets to **0**, the purer the group.

and then information gain calculated

Entropy = Confusion 🤔

Information Gain = Confusion reduced 🧹

Decision Tree = Keep asking questions until the groups become clear. 🌳

$$Information\ Gain = Entropy_{parent} - Weighted\ Entropy_{children}$$

 Tree 1 —→ PASS
 Tree 2 —→ PASS
 Tree 3 —→ FAIL
 Tree 4 —→ PASS
 Tree 5 —→ PASS



 Majority Vote



 PASS

Many trees together =
 Random
Forest

"Random Forest is a team of decision trees. Each tree gives an answer, and the forest takes the majority vote." 🌳🌳🌳🗳

 Why is it called
"Random"?

The algorithm introduces randomness when creating the trees.

That's why:

Random + Forest = Random Forest  



Good Question!

Why is it called **Random Forest**?



Random Forest doesn't give every tree the same information. Each tree looks at a **random subset of features** (different inputs). This introduces randomness when creating the trees.

Tree 1



This tree looks at:



Study Hours



Attendance



Makes a prediction

Tree 2



This tree looks at:



Previous Marks



Assignments



Makes a prediction

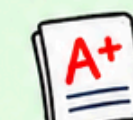
Tree 3



This tree looks at:



Attendance



Previous Marks



Makes a prediction

Tree 4



This tree looks at:



Study Hours



Assignments



Makes a prediction

All trees make a prediction

Take the majority vote

Final Prediction

Different trees see different parts of the data.

This randomness helps the model be more accurate and less likely to make mistakes.

Random + Forest = Random Forest

Random forest source code

DATA

↓

Create many Decision Trees

↓

Each tree makes a prediction

↓

Collect predictions

↓

Majority Voting

↓

FINAL PREDICTION

```
import numpy as np
```

```
import pandas as pd
```

```
# -----
```

```
# 1. DATASET
```

```
# -----
```

```
data = {
```

```
    "Age": [22, 25, 28, 35, 40, 45, 50, 23, 30, 38],
```

```
    "Visits": [2, 5, 6, 8, 7, 3, 2, 6, 7, 9],
```

```
    "Time_Spent": [5, 15, 20, 35, 30, 10, 8, 25, 28, 40],
```

```
    "Previous_Purchases": [0, 1, 2, 4, 3, 1, 0, 2, 3, 5],
```

```
    "Bought": [0, 0, 1, 1, 1, 0, 0, 1, 1, 1]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
# -----
```

```
# 2. TREE 1
```

```
# -----
```

```
def tree1(visits):
```

```
    if visits > 5:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
# -----
```

```
# 3. TREE 2
```

```
# -----
```

```
def tree2(time_spent):
```

```
    if time_spent > 20:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
# -----
```

```
# 4. TREE 3
```

```
# -----
```

```
def tree3(previous_purchases):
```

```
    if previous_purchases > 1:
```

```
        return 1
```

```
    else:
```

```
        return 0
```

```
# -----
```

```
# 5. RANDOM FOREST
```

```
# -----
```

```
def random_forest(visits, time_spent, previous_purchases):
```

```
    prediction1 = tree1(visits)
```

```
    prediction2 = tree2(time_spent)
```

```
    prediction3 = tree3(previous_purchases)
```

```
    predictions = [
```

```
        prediction1,
```

```
        prediction2,
```

```
        prediction3
```

```
    ]
```

```
print("\nTree Predictions:")  
  
print("Tree 1:", prediction1)  
  
print("Tree 2:", prediction2)  
  
print("Tree 3:", prediction3)
```

```
# Majority voting  
  
final_prediction = max(  
    set(predictions),  
    key=predictions.count  
)  
  
return final_prediction
```

```
# -----
```

```
# 6. NEW CUSTOMER
```

```
# -----
```

```
result = random_forest(  
    visits=7,  
    time_spent=25,  
    previous_purchases=3  
)
```

```
# -----
```

```
# 7. FINAL RESULT
```

```
# -----
```

```
print("\nFinal Prediction:")
```

```
if result == 1:
```

```
    print("Customer is likely to BUY 🛒")
```

```
else:
```

```
    print("Customer is unlikely to BUY ❌")
```